Manipal School of Information Sciences

Cryptology Lab
LAB ASSIGNMENT 02

# Importance of Cryptography

## HTTP and HTTPS Traffic Analysis Using Wireshark

| | |
|---|---|
| Student Name: | Ribhav Devprasad |
| Reg. No.: | 261100690002 |
| Date: | 19 August 2026 |

# Contents


| Section | Title |
|---|---|
| 1 | Problem Statement |
| 2 | Experimental Setup |
| 3 | Experiment Procedure |
| 4 | HTTP Login Page and Capture |
| 5 | HTTPS Login Page and Capture |
| 6 | Comparison of Observations |
| 7 | Role of Encryption |
| 8 | Safety, Limitations and Conclusion |


# 1 Problem Statement

Using Wireshark, investigate the transmission of dummy login credentials over HTTP and HTTPS in a controlled environment. Analyze the captured network traffic, compare how the credentials are transmitted in each protocol, and explain the role of encryption in protecting sensitive information.

## Aim

To demonstrate, using packet evidence, that HTTP exposes login data as plaintext while HTTPS protects the same application data with TLS encryption.

## Objectives

The experiment creates a local test login, captures the HTTP and HTTPS trials, records the login page and packet-level observations, compares visible and protected fields, and states the security implications.

> Only dummy credentials and a private test server are used. No real passwords, public websites, or unauthorized networks are involved.

# 2 Experimental Setup

The experiment uses two virtual machines. Ubuntu provides the local web server, and Kali opens the login page and records the traffic with Wireshark. Both systems are connected through the same Host-Only or Bridged virtual network.

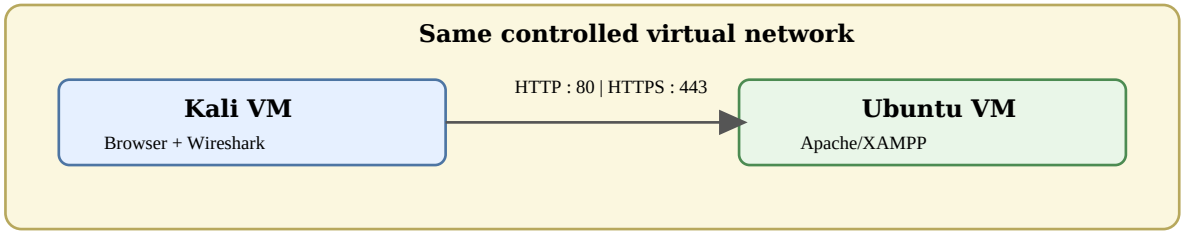



Figure 1. Controlled two-VM setup used for the lab.

| Component | Role | Configuration |
|---|---|---|
| Ubuntu VM | Web server | Apache/XAMPP; local login page; ports 80 and 443 |
| Kali VM | Client and observer | Browser/curl and Wireshark |
| Credentials | Test data | Username: admin; password: SuperSecret123 |
| Network | Isolation | Host-Only or Bridged Adapter; no production traffic |

Table 1. Experimental setup.

# 3 Experiment Procedure

First, identify the non-loopback address of each virtual machine and verify that the two machines can communicate. In this lab, the Ubuntu VM is addressed as 10.0.2.10. On Ubuntu, create the local login page and start the server. Then create a self-signed certificate for the HTTPS endpoint. The warning shown by the browser is expected for a local certificate.

```
ip -brief addr
ping -c 3 10.0.2.10
sudo /opt/lampp/lampp start
```

```
ip route get 10.0.2.10
Capture filter: host 10.0.2.10
HTTP filter: http.request.method == "POST"   HTTPS filter: tls
```

# 4 HTTP Login Page and Capture

The first trial uses the local HTTP address. The browser displays the login page and submits the dummy credentials without TLS protection.

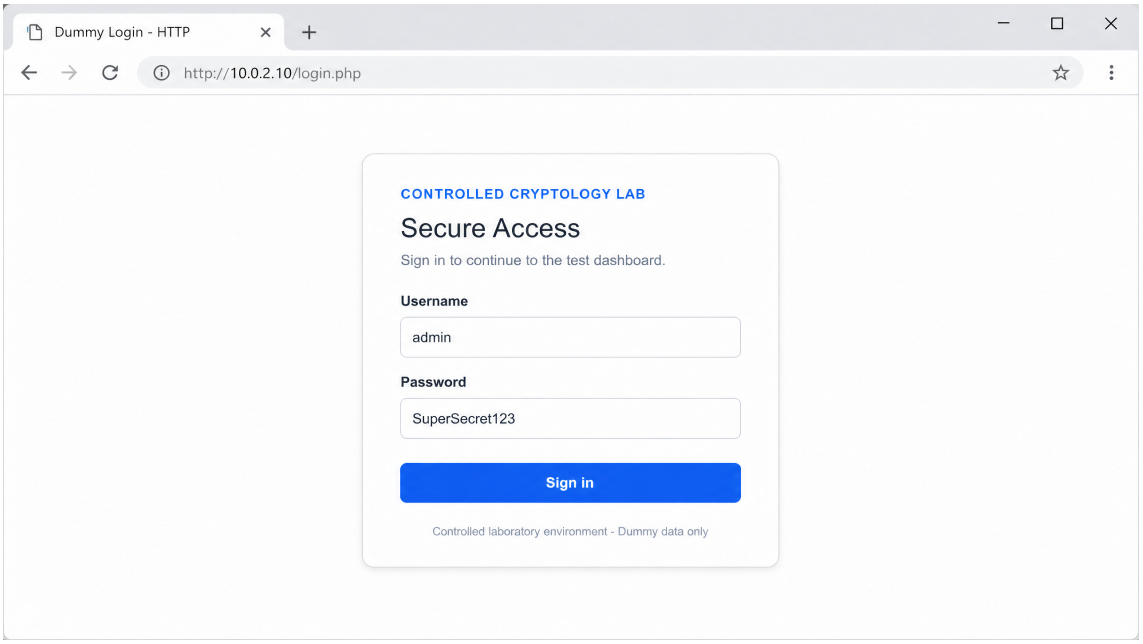



Figure 2. Dummy login page opened over HTTP.

The login page is intentionally simple. Its purpose is to generate a predictable POST request that can be inspected in Wireshark.

### HTTP packet-list evidence

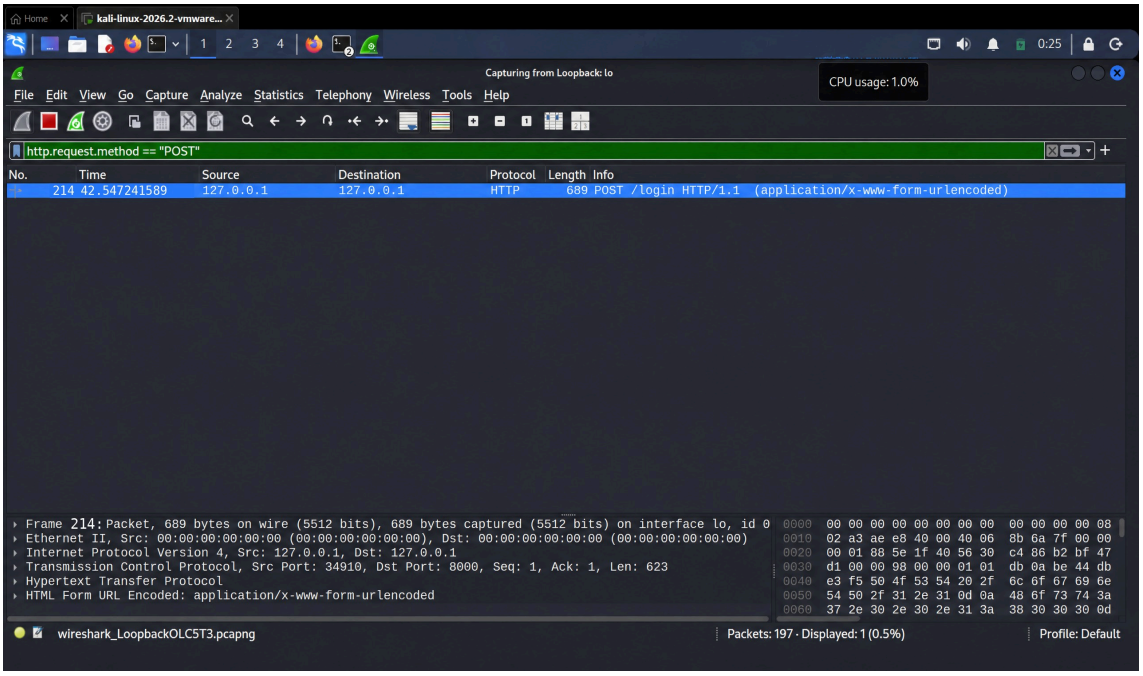



Figure 3. Wireshark display filter selects the HTTP POST request.

The selected packet is an HTTP POST. The request is not protected by TLS, so the form data can be inspected directly.

# 4.1 HTTP Credential Evidence

The packet details identify the form fields and the HTTP stream contains the complete conversation. The following evidence is generated from the controlled dummy login capture.

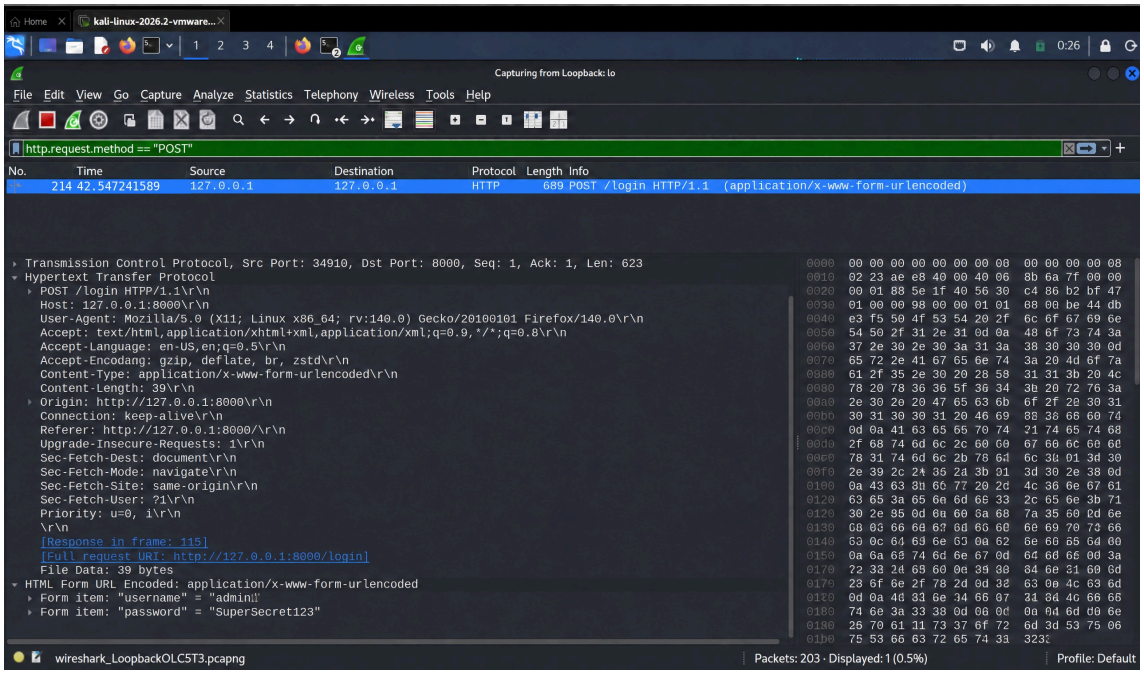



Figure 4. Native Wireshark Follow HTTP Stream window showing the plaintext HTTP request, dummy credentials, redirect, and session cookie.

| Observed field | HTTP result |
|---|---|
| Username | Readable as admin |
| Password | Readable as SuperSecret123 |
| Server response | 302 redirect and Set-Cookie are readable |
| Security meaning | A passive observer can copy the credentials and session information |

Table 2. HTTP credential observations.

# 5 HTTPS Login Page and Capture

The second trial repeats the same login over the local HTTPS address. The credentials are unchanged; only the transport protection is different.

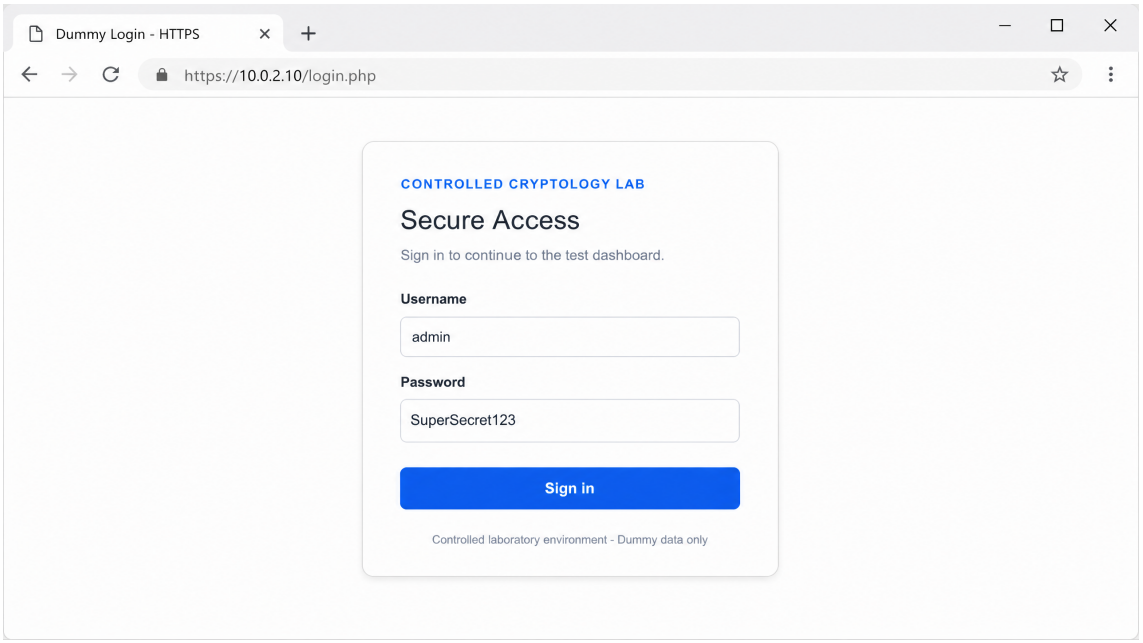



Figure 5. The same dummy login page opened over HTTPS.

After the local certificate warning is accepted, the same form is submitted. Wireshark shows TLS records instead of readable HTTP request text.

## HTTPS/TLS packet evidence

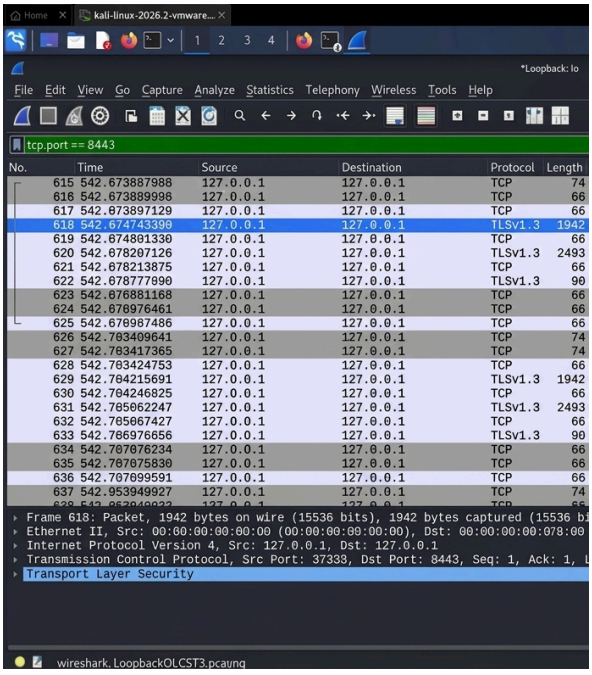



Figure 6. TLS handshake and Application Data records in Wireshark.

## 5.1 HTTPS Encrypted Application Data

The TLS handshake is visible because the browser and server must negotiate a protected session. The actual login is carried inside TLS Application Data.

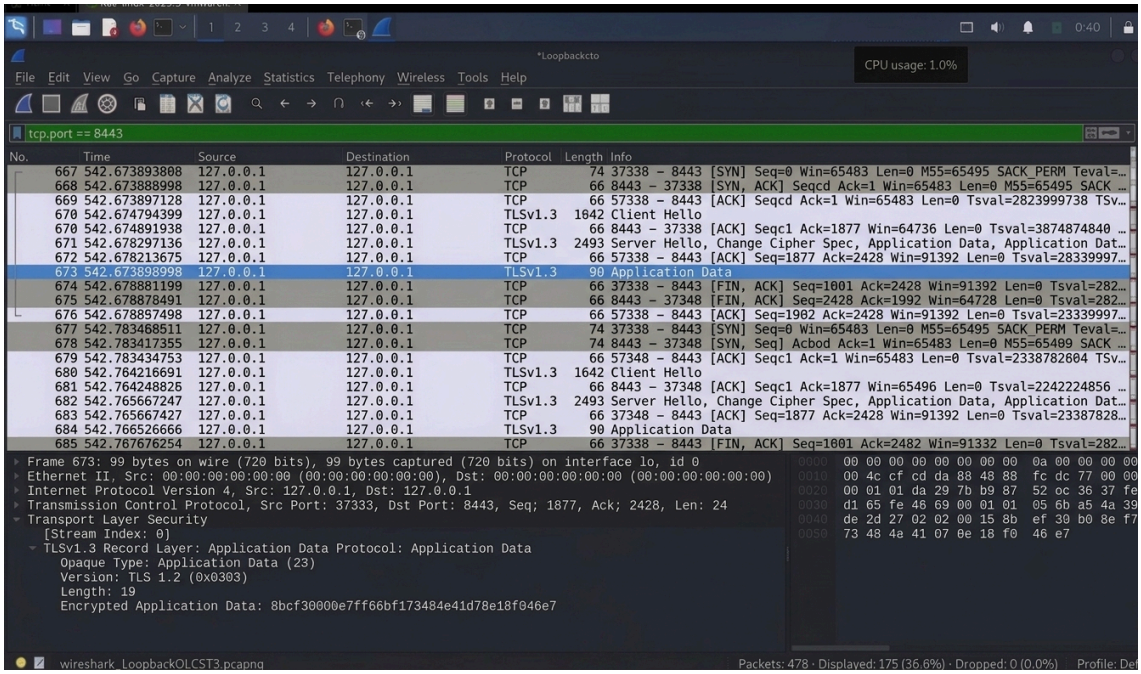



Figure 7. Native Wireshark packet-detail view showing TLS Application Data and encrypted bytes; the plaintext password is not visible.

A search for the plaintext password returns no match in the HTTPS capture. The network may still reveal IP addresses, timing, traffic size, and some handshake metadata, but the sensitive application payload is not readable.

| Observed field | HTTPS result |
|---|---|
| Username | Not readable in ordinary packet bytes |
| Password | Not readable in ordinary packet bytes |
| Login request | Contained inside TLS Application Data |
| Security meaning | Passive packet inspection reveals ciphertext rather than credentials |

Table 3. HTTPS observations.

# 6 Comparison of Observations

The two trials use the same login form and the same dummy values. The following table separates what remains visible at the connection level from what is protected at the application level.

| What an observer sees | HTTP | HTTPS |
|---|---|---|
| Destination port | 80 | 443 |
| IP addresses and timing | Visible | Visible |
| HTTP method and path | Visible | Inside encrypted data |
| Username | Visible | Not readable |
| Password | Visible | Not readable |
| Session cookie | Potentially visible | Protected by TLS |
| Page content | Potentially visible | Protected by TLS |
| Application payload | Plaintext | Ciphertext |

Table 4. HTTP and HTTPS comparison.

## Interpretation

HTTP provides no confidentiality for the application payload. HTTPS protects the payload through TLS encryption and also provides integrity protection. HTTPS does not make the connection itself invisible, but it prevents a passive observer from reading the login data directly from the captured packets.

# 7 Role of Encryption

Encryption converts plaintext into ciphertext using cryptographic keys. In this experiment, the HTTP form body can be read immediately, while the HTTPS form body is transformed into TLS-protected application data before transmission.

| Security property | Contribution in this lab |
|---|---|
| Confidentiality | Protects the username, password, cookie, and page content from passive observation. |
| Integrity | Helps detect unauthorized modification of data in transit. |
| Authentication | The certificate helps the client identify the intended server; a self-signed certificate is suitable only for this controlled lab. |

Table 5. Role of encryption.

# 8 Safety, Limitations and Conclusion

The experiment is limited to a local test server, a private virtual network, and dummy data. It must not be performed against public systems or other people’s traffic. After the experiment, stop the server and remove the test files. A client-owned TLS key log can decrypt a capture for debugging, but that requires access to the endpoint session keys and does not mean a passive observer can decrypt ordinary HTTPS traffic.

The investigation confirms that HTTP transmits credentials in readable form, whereas HTTPS carries the same credentials inside encrypted TLS Application Data. Encryption is therefore essential for protecting sensitive information during network communication. HTTPS should still be combined with secure password storage, valid certificates, strong authentication, and correct server configuration.